



CS100 Recitation 3

Control Flow, Preprocessing, Variables Explained

@zhangyzh2022@shanghaitech.edu.cn

@lichf2025@shanghaitech.edu.cn

@heyx2025@shanghaitech.edu.cn

Contents

Contents

- 1 控制流 (Control Flow): 条件语句 & 循环 ->
- 2 预处理指令 (Preprocessor Directives): `#include` & `#define` ->
- 3 变量 ->

1. Control Flow

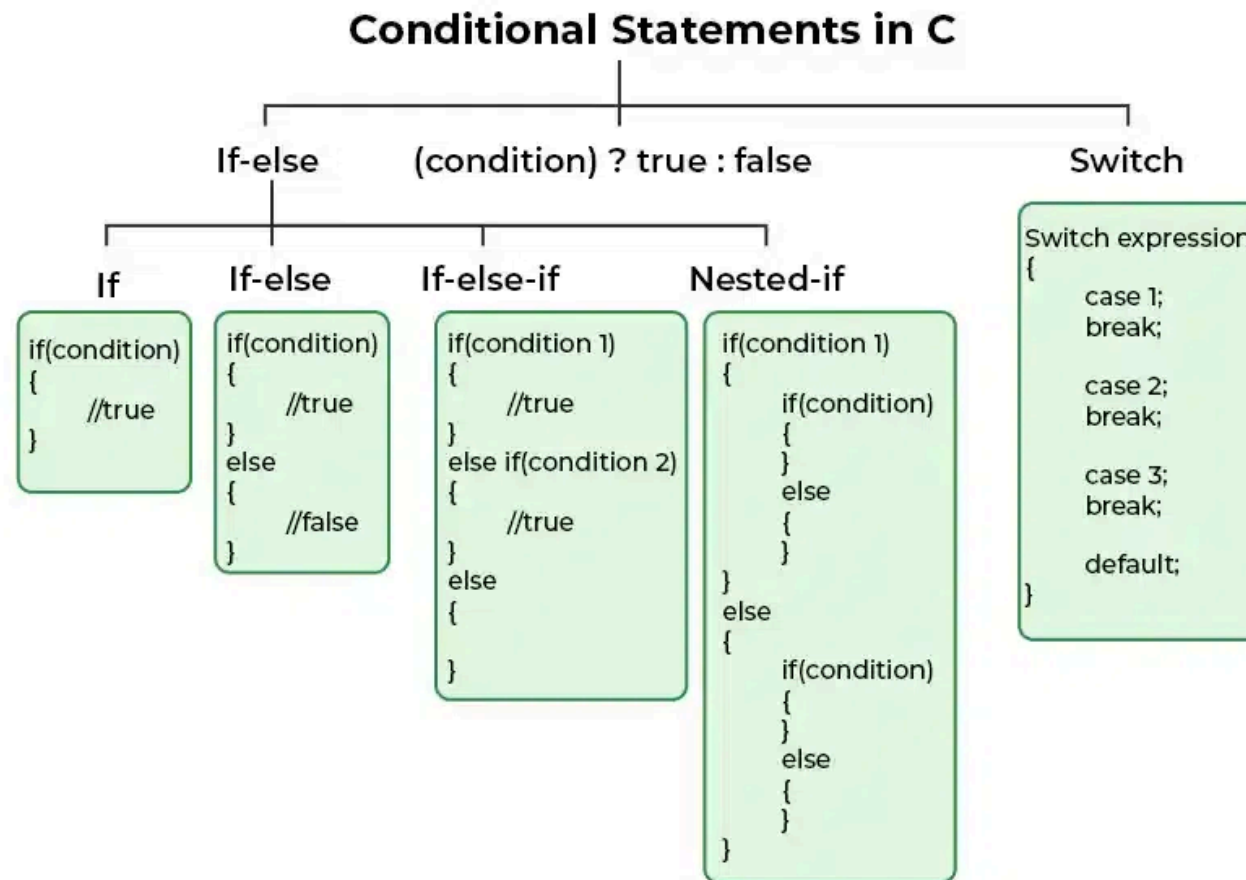
1. Control Flow

1.1. Overview Of the Section

- 条件语句 (Conditional Statements)
 - if, if- else, if-else if, nested(嵌套)-if
 - switch-case
- 循环 (Loops)

1. Control Flow

1.2. Conditional Statements



1. Control Flow

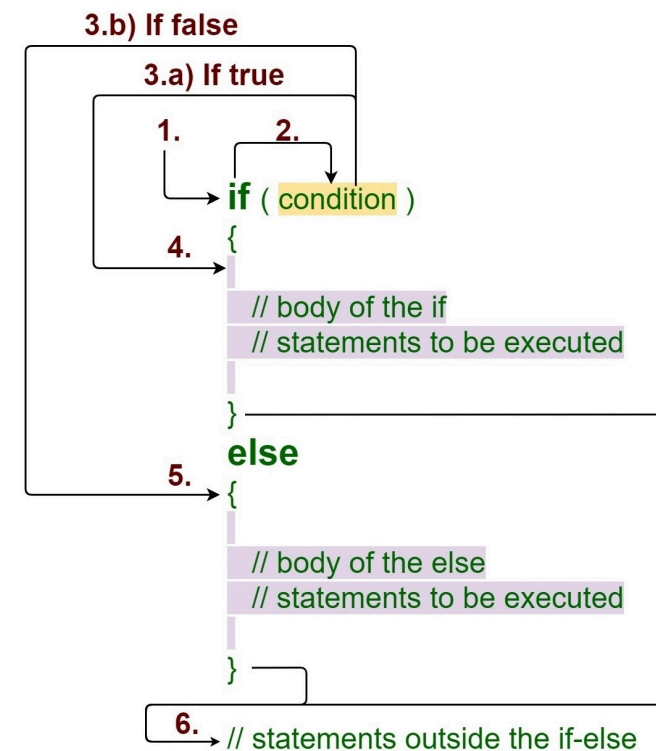
1.2. Conditional Statements `if-else`

Syntax:

```
if (condition) statement_T else statement_F
```

condition: 一个标量类型（算术类型或指针类型）的**表达式**。

- 算术类型 $\rightarrow$: 包括整数类型（Integer types）和浮点数类型（floating-point types）。
 - 示例: `if (a > 0)`
- 指针类型 $\rightarrow$: 指向对象（objects）、函数（functions）等的指针。
 - 示例: `if (ptr) {...}` 等价于 `if (ptr != NULL)`。



1. Control Flow

Exercise 1

```
#include <stdio.h>

int i;

int main(void) {
    if (i) {
        // Do nothing
    }
    else {
        printf("Else");
    }
    return 0;
}
```

Question: What is correct about the above program?

1. 执行 `if` 代码块。
2. 执行 `else` 代码块。
3. 由于 `i` 未初始化，结果不可预测。
4. Error: `else` 的位置不正确。

1. Control Flow

Exercise 1

```
#include <stdio.h>

int i;

int main(void) {
    if (i) {
        // Do nothing
    }
    else {
        printf("Else");
    }
    return 0;
}
```

Question: What is correct about the above program?

1. 执行 `if` 代码块。
2. 执行 `else` 代码块。
3. 由于 `i` 未初始化，结果不可预测。
4. Error: `else` 的位置不正确。

“ 变量*i*为全局变量，默认初始化为0，因此执行else代码块，输出"Else"。

1. Control Flow

Syntax:

```
switch (expression)
{
    case constant1:
        // statements
        break;
    case constant2:
        // statements
        break;
    ...
    default:
        // default statements
}
```

1.2. Conditional Statements **switch-case**

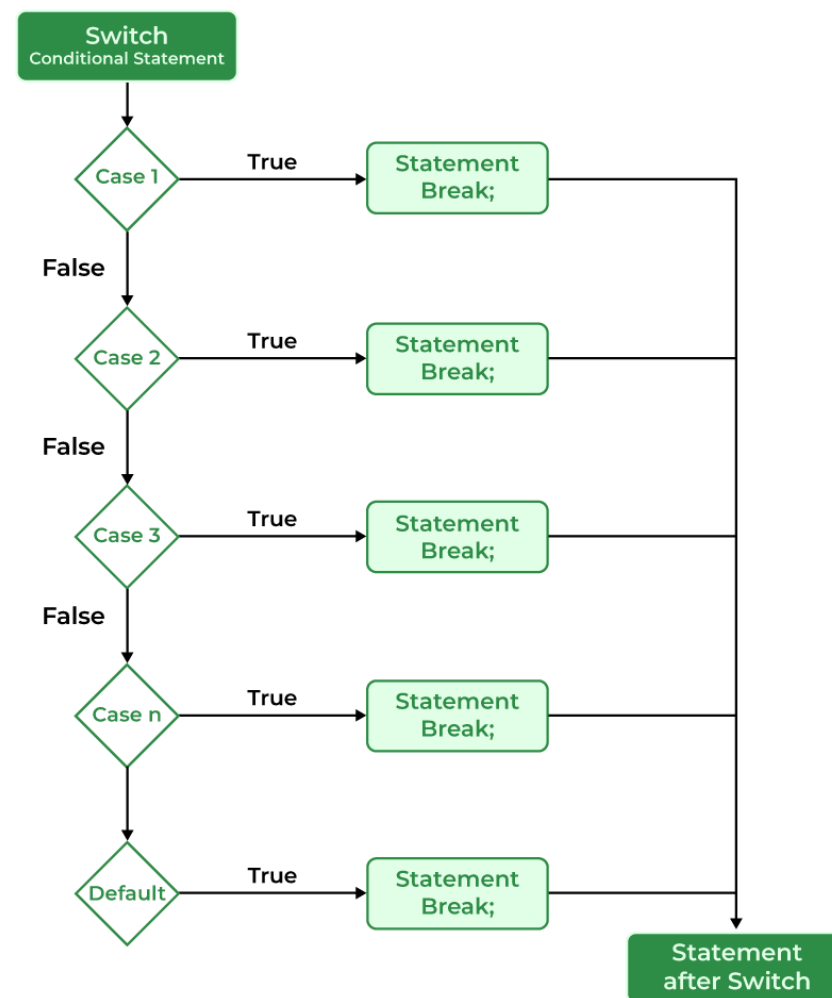
- **Expression**: 任何整数类型的表达式 (char, signed/unsigned int, Enum)
- **case** 标签中的表达式:
 - Integer Constants Expression ->
 - Integer Constants ->: 42
 - Character Constants ->: a
 - true, ...
- **constant** 的值在编译时是已知的。

1. Control Flow

1.2. Conditional Statements `switch-case`

一旦 Control Flow 跳转到与 `case` 标签对应的语句，它会继续**向下执行**，直到遇到 `break;` 或达到 `switch` 块的末尾。

- 如果在下一个 `case` 之前没有 `break;`，则控制流程将**不会停止**。



1. Control Flow

Exercise 2: What the output of the following program?

```
#include <stdio.h>
void func(int x){
    printf("func(%d): ", x);
    switch(x){
        case 1: printf("case 1, ");
        case 2: printf("case 2, ");
        case 3: printf("case 3.\n"); break;
        case 4: printf("case 4, ");
        case 5:
        case 6: printf("case 5 or case 6, ");
        default: printf("default.\n");
    }
}
int main(void){
    for(int i = 1; i < 9; ++i) func(i);
}
```

1. Control Flow

Exercise 2

Output:

```
func(1): case 1, case 2, case 3.  
func(2): case 2, case 3.  
func(3): case 3.  
func(4): case 4, case 5 or case 6, default.  
func(5): case 5 or case 6, default.  
func(6): case 5 or case 6, default.  
func(7): default.  
func(8): default.
```

1. Control Flow

1.3. switch-case: Scope (作用域)

Block Scope ->:

- 由 if、switch、for、while、do-while、函数定义等建立。
- 作用域从声明点开始，到声明它的代码块或语句的末尾结束。
- 通常，代码块通过 {} 标记。

1. Control Flow

1.3. Scope (作用域)--the block is maked by {}

```
#include <stdio.h>
int main(void)
{
    {
        int x = 1;
    }
    printf("%d", x);
    return 0;
}
```

1. Control Flow

1.3. Scope (作用域)--the block is maked by {}

```
heaticy@heaticy-ThinkStation-P320 ~/CS100 (master) [1]> gcc a.c -o a.out
```

```
a.c: In function 'main' :
```

```
a.c:7:18: error: 'x' undeclared (first use in this function)
```

```
7 |      printf("%d", x);  
  |                  ^
```

```
a.c:7:18: note: each undeclared identifier is reported only once for each function it appears in
```

1. Control Flow

1.3. switch-case: Scope

```
switch (expr) {  
  case 1:  
    int x = 42;  
    do_something(x, expr);  
  case 2:  
    printf("%d\n", x);  
}
```

- What happens if `expr` evaluates to 2?

1. Control Flow

1.3. switch-case: Scope

- 如果 `expr == 2`, 控制流会跳过 `int x = 42;`, 但由于名称查找规则, `x` 仍然可见。
 - `x` 的声明仍然在 `switch` 作用域内, 因为它位于 `{}` 块中。
- 为了避免这种情况, 应在 `case` 中为变量声明使用单独的代码块。
 - 简而言之, 使用 `{}` 来限制作用域。

1. Control Flow

1.3. switch-case: Scope

```
switch (expr) {  
  case 1: { // Use {} to limit `x` to this block.  
    int x = 42;  
    do_something(x, expr);  
  }  
  case 2:  
    printf("%d\n", x); // Error: `x` is not declared in this scope.  
}
```

1. Control Flow

Exercise 3

```
# include <stdio.h>
int main(void) {
    int i = 0;
    for (i = 0; i < 20; i++){
        switch(i){
            case 0: i += 5;
            case 1: i += 2;
            case 5: i += 5;
            default: i += 4;
            break;
        }
        printf("%d\n", i);
    }
    return 0;
}
```

Question: What is the output?

- 5 10 15 20
- 7 12 17 22
- 16 21
- Compile Error

1. Control Flow

Exercise 3

```
# include <stdio.h>
int main(void) {
    int i = 0;
    for (i = 0; i < 20; i++){
        switch(i){
            case 0: i += 5;
            case 1: i += 2;
            case 5: i += 5;
            default: i += 4;
                break;
        }
        printf("%d\n", i);
    }
    return 0;
}
```

Question: What is the output?

- 5 10 15 20
- 7 12 17 22
- **16 21**
- Compile Error

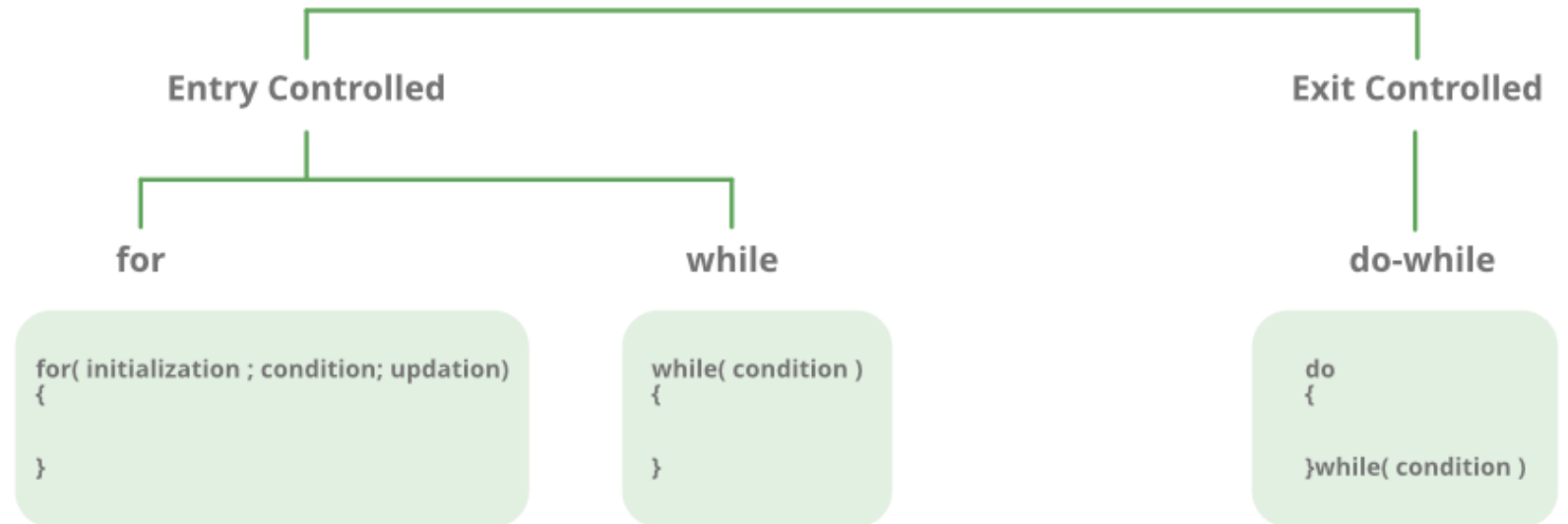
“最初， $i = 0$ 。由于 case 0 为真， i 变成了 5，并且由于在 switch 块中的 case 语句后没有 break 语句，程序会继续执行直到 switch 块的最后一条语句， i 变成了 16。然后在下一次迭代中，没有任何 case 为真，所以程序执行到了 default， i 变成了 21。

1. Control Flow

1.4. Loops

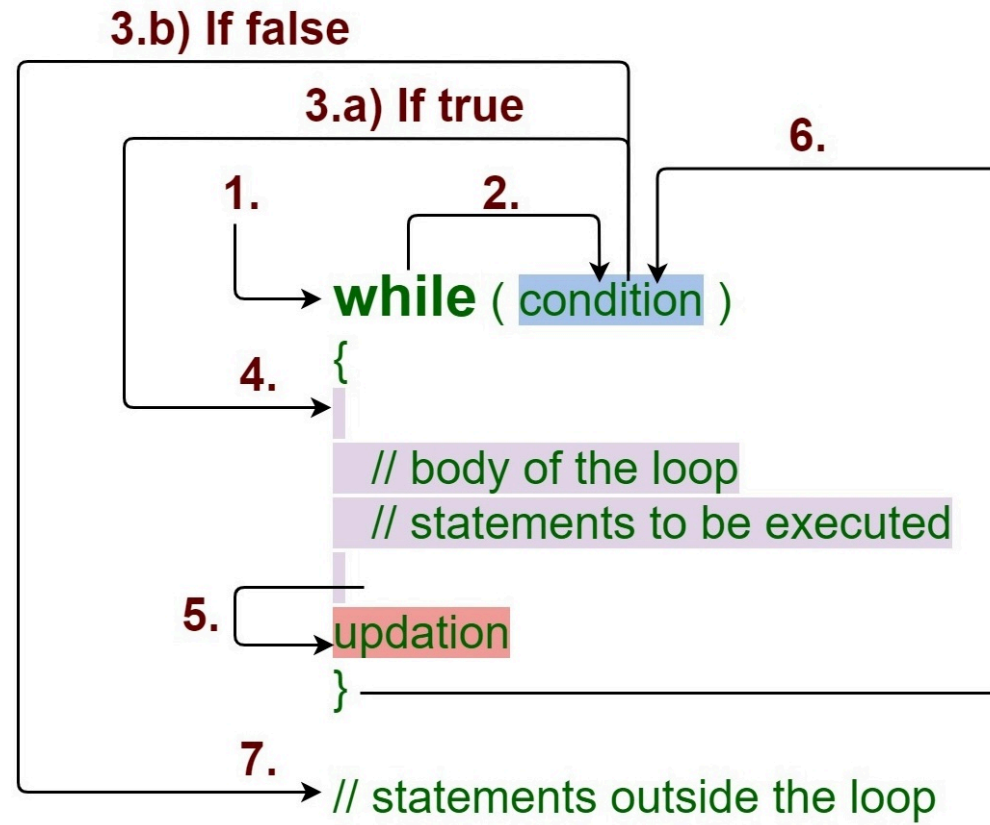
Loops 的主要语法有三种:

- while
- for
- do - while



1. Control Flow

1.4. Entry-controlled Loop: `while`



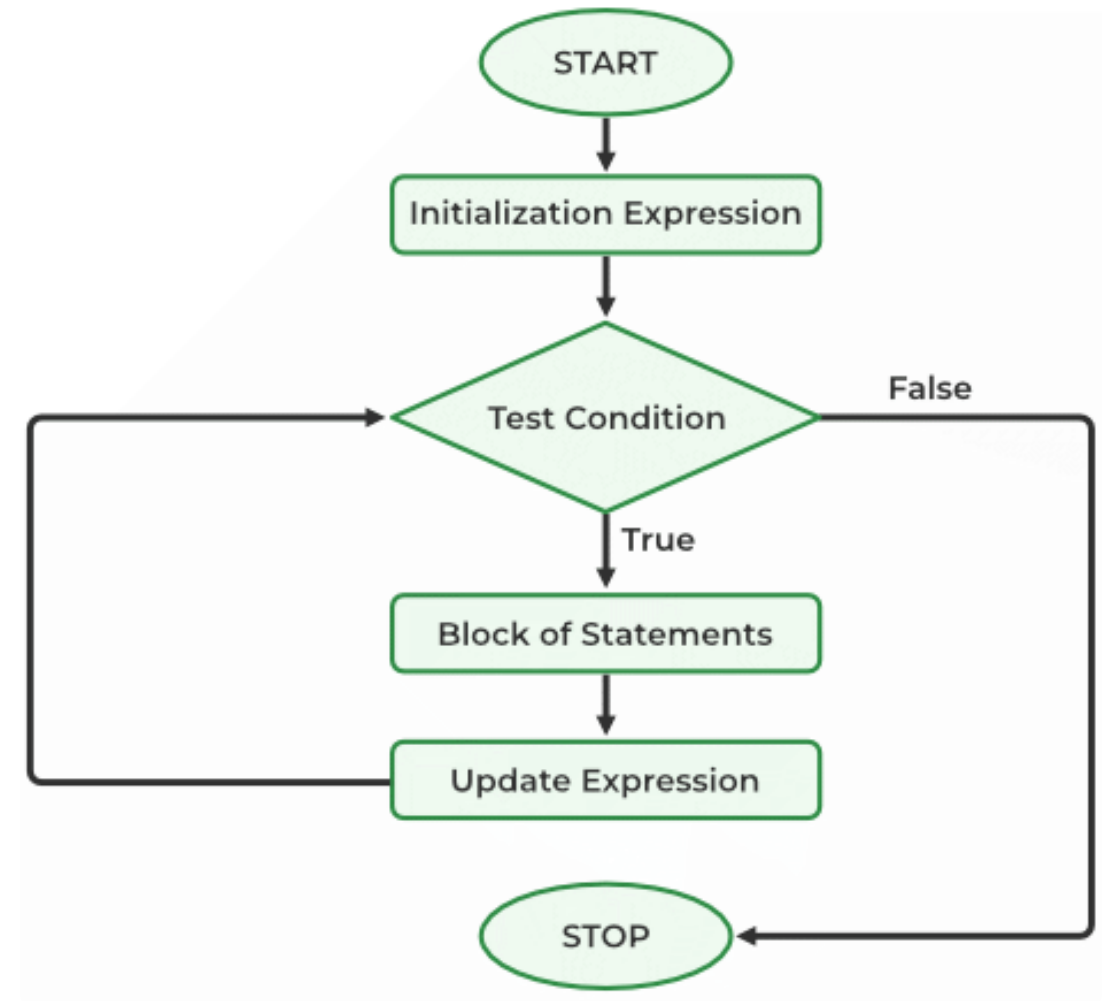
1. Control Flow

1.4. Entry-controlled Loop: for

`for ( int i = 5 ; i <= 10 ; i ++ )`

Initialization Test Condition Updation

- `init_clause`: **expression**/declaration
- `expression`: same to `while`
- `for(;;){...}` = `while(1){...}`



1. Control Flow

Exercise 4

```
#include <stdio.h>

int main(void){
    int i = 1024;
    for (; i; i >>= 1)
        printf("CS100 is easy if you work hard.\n");
    return 0;
}
```

Question: 在上述程序中, `CS100 is easy if you work hard.` 会被 print 多少次?

1. Control Flow

Exercise 4

```
#include <stdio.h>

int main(void){
    int i = 1024;
    for (; i; i >>= 1)
        printf("CS100 is easy if you work hard.\n");
    return 0;
}
```

Question: 在上述程序中, `CS100 is easy if you work hard.` 会被 print 多少次?

“ 11次。1024的二进制为10000000000, 因此右移11次后变为0, 循环终止。

1. Control Flow

Exercise 5

```
#include <stdio.h>
int main(void){
    int n;
    for (n = 9; n != 0; n--)
        printf("n = %d\n", n--);
    return 0;
}
```

Question: What is the output?

- 9 7 5 3 1
- 9 8 7 6 5 4 3 2 1
- Infinite loop
- 9 7 5 3

1. Control Flow

Exercise 5

```
#include <stdio.h>
int main(void){
    int n;
    for (n = 9; n != 0; n--)
        printf("n = %d\n", n--);
    return 0;
}
```

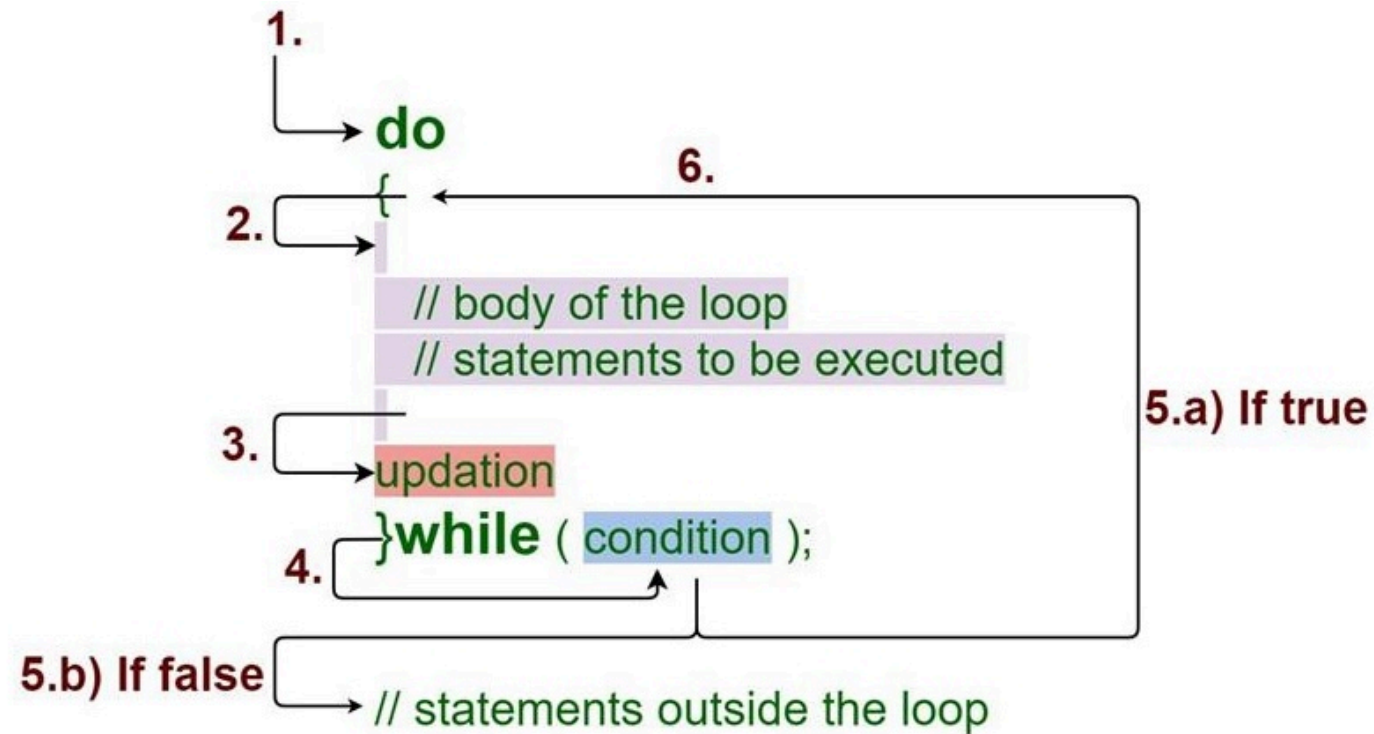
Question: What is the output?

- 9 7 5 3 1
- 9 8 7 6 5 4 3 2 1
- **Infinite loop**
- 9 7 5 3

“ Infinite loop.
每次循环都会-2，因此n永远不会等于0。

1. Control Flow

Exit-controlled Loop: **do-while**



- 循环体至少执行一次。

1. Control Flow

Exercise 6

```
#include <stdio.h>
int main(void){
    int c = 5, n = 10000;
    do{
        n /= c;
    } while(c--);

    printf("%d\n", n);
    return 0;
}
```

Question: What is the output?

1. 1
2. 0
3. Compile Error
4. Undefined Behavior

1. Control Flow

Exercise 6

```
#include <stdio.h>
int main(void){
    int c = 5, n = 10000;
    do{
        n /= c;
    } while(c--);

    printf("%d\n", n);
    return 0;
}
```

Question: What is the output?

1. 1
2. 0
3. Compile Error
4. **Undefined Behavior**

“ Undefined Behavior.
int类型除以0会导致未定义的行为。

2. Preprocessor Directives

(预处理指令)

2. Preprocessor Directives (预处理指令)

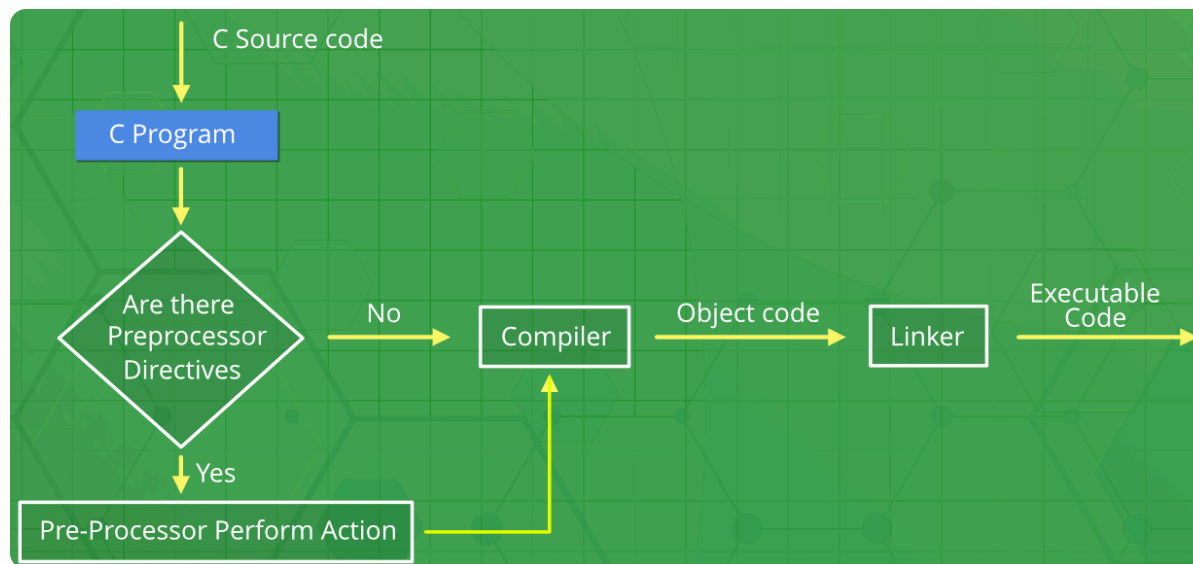
2.1. Overview Of Preprocessor Directives

- 预处理器 (Preprocessor)
- 预处理指令 (Preprocessor Directives)

2. Preprocessor Directives (预处理指令)

2.2. Preprocessor & Preprocessor Directives

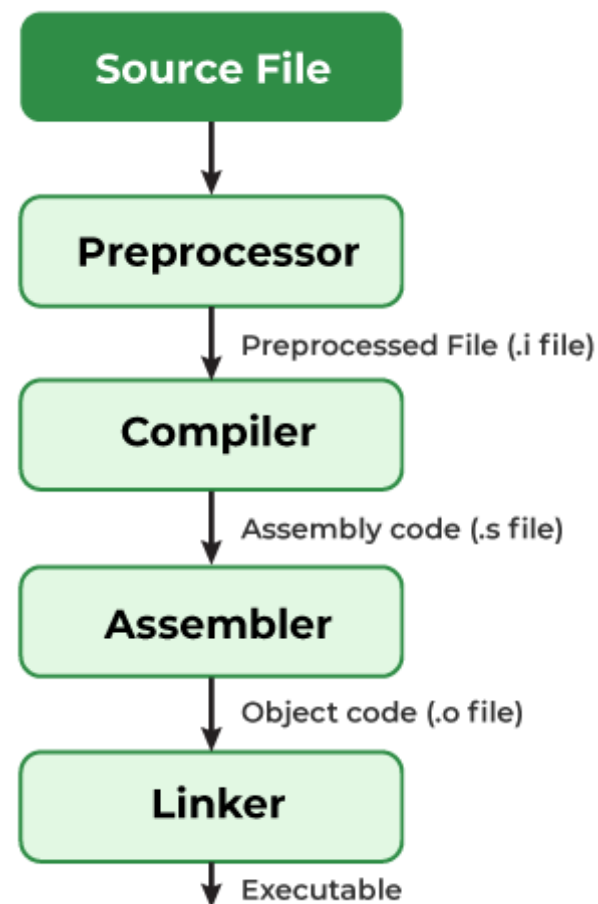
- 在编译 C/C++ 程序时，编译器首先会调用 **预处理器** 来处理所有的预处理指令。
 - 有时预处理器被认为属于编译器的一部分。
- 预处理指令** 控制预处理器的行为。



2. Preprocessor Directives (预处理指令)

Exercise 7: Try the following commands.

- `gcc a.c --save-temps`
 - 永久保存中间文件。
- `gcc a.c -E: GCC Options ->`
 - `-E` 告诉 GCC 在预处理阶段结束后停止。
 - 它将预处理后的源代码输出到控制台。



2. Preprocessor Directives (预处理指令)

2.2. 预处理指令 - 控制预处理器

Examples:

- **Source file inclusion:** `#include`
- **Replacing text macros (宏):** `#define`, `*#undef` (cancels a previous definition by `#define`)
 - **Macro:** 程序中一段代码, 该代码会被 Macro 的 Value 所替换
- **Conditional inclusion:** `#if`, `#ifdef`, `#ifndef`, `#else`, etc.
- ***实现定义行为控制:** `#pragma`
- ***文件名和行信息:** `#line` (非常有用), ***Diagnostic directives:** `#error`, etc.

2. Preprocessor Directives (预处理指令)

2.3. Source File Inclusion: #include

Syntax:

```
#include <filename> (newline)
#include "filename" (newline)
```

`#include` 指令在 C 语言编程中用于**文本替换**。它会找到指定的文件，将**该文件的内容插入到源文件中指令所在的位置**。

2. Preprocessor Directives (预处理指令)

2.3. Source File Inclusion: `#include`

- `#include` 和 `import` 的区别:
`import` 是其他编程语言（如 Python 和 Java）中的关键字，用于引入模块或包。它通常包含一些额外的功能，例如命名空间管理和模块加载。**C++ 自 C++20 标准开始支持模块** ->。

2. Preprocessor Directives (预处理指令)

2.4. Search Path of `#include`

- `#include <file>`: 这种形式通常用于**标准库头文件**。
 - **默认行为 (GCC 预处理器)**: 它只在标准系统目录中搜索, 这些目录是预先配置的, 可能因系统和 GCC 配置而异。
 - **命令行选项**可以用来将额外的目录添加到搜索路径中。最常用的选项是 `-I`, 可以如下指定: `gcc a.c -o a -I/home/liuxiao/my_awesome_library/include`

2. Preprocessor Directives (预处理指令)

2.4. Search Path of `#include`

- `#include "file"`: 这种形式通常用于用户自定义的头文件。
 - 预处理器首先在当前文件所在的目录中查找，将文件视为相对路径，然后继续在标准系统目录中查找。
 - 示例: `#include "../my_library/include/my_header.h"`

2. Preprocessor Directives (预处理指令)

2.5. Replacing text macros: #define

```
#define N 1000  
#define MAX(A, B) A < B ? B : A
```

之后，所有出现的 `N` 将被替换为 `1000`，所有出现的 `MAX(expr1, expr2)` 将被替换为 `expr1 < expr2 ? expr2 : expr1`。

- 形式 `#define MACRO_NAME text_to_be_replaced` 被称为 **对象类宏**。它将每个定义过的 `MACRO_NAME` 替换为 `text_to_be_replaced`。
- 另一种形式 `#define MACRO_NAME (parameters, ...) text_to_be_replaced` 被称为 **函数类宏**。

注意：重新定义先前定义的宏是不合法的，除非新定义与旧定义完全相同。这意味着 **两个文件**，如果都定义了某个宏，则该 **定义必须一致**。

2. Preprocessor Directives (预处理指令)

2.6. #undef

#undef 指令具有以下语法：

```
#undef MACRO_NAME
```

此指令通过 #define 取消标识符的先前定义。

2. Preprocessor Directives (预处理指令)

2.7. #define: Scope

- 宏从其 `#define` 指令处开始**生效**。
- 当它遇到对应的 `#undef` 指令或到达当前文件末尾时，将**失效**。

```
#include <stdio.h>

#define APPLE (10 + PEAR)

int main(void){
#define PEAR 15
    int x = APPLE;
    printf("x is %d.\n", x);
    // x is 25.
#undef PEAR

#define PEAR 100
    int y = APPLE;
    printf("y is %d.\n", y);
    // y is 110.
}
```

2. Preprocessor Directives (预处理指令)

2.7. #define: Nested Definitions

1. 如果一个宏的名称出现在它自身的替换文本中，则**不会被替换**。
2. 这些未被替换的宏名称随后将**无法用于进一步的替换**。

```
#include <stdio.h>

#define APPLE (10 + APPLE) // 1)

void foo(void){
    int y = APPLE; // 2)
    // y is undeclared; error
#undef APPLE
#define APPLE 15
}

int main(void){
    int x = APPLE;
    printf("x is %d.\n", x);
    // x is 15 if '2)' is commented out.
}
```

2. Preprocessor Directives (预处理指令)

2.7. #define: Function-Like Form

回顾：在第一张关于 `#define` 的幻灯片中，我们介绍了函数式的宏 `MAX(A, B)`：

```
#define MAX(A, B) A < B ? B : A
```

```
#include <stdio.h>

// Macro definition
#define AREA(l, b) (l * b)

int main(void){
    // Given lengths l1 and l2
    int l1 = 10, l2 = 5, area;

    // Find the area using macros
    area = AREA(l1, l2);

    // Print the area
    printf("%d\n", area);

    return 0;
}
```

2. Preprocessor Directives (预处理指令)

Exercise 8

根据以下用法，评估我们的宏定义是否正确。你能发现哪些潜在问题？如何解决？

```
#define MAX(A, B) A < B ? B : A
```

```
int i = 10, j = 15;  
int k = MAX(i, j) + 1;
```

- 对于这个定义呢？`#define MAX(A, B) (A < B ? B : A)`
 - 提示：评估 `MAX(i, i & j)` 的结果。
- 对于这个定义呢？`#define MAX(A, B) ((A) < (B) ? (B) : (A))`
 - 提示：同样尝试使用 `MAX(i, ++j)`。

2. Preprocessor Directives (预处理指令)

Exercise 8

根据以下用法，评估我们的宏定义是否正确。你能发现哪些潜在问题？如何解决？

```
#define MAX(A, B) A < B ? B : A
```

```
int i = 10, j = 15;  
int k = MAX(i, j) + 1;
```

“ 宏展开后，k 的计算变为 $i < j ? j : i + 1$ ，条件运算符的优先级低于加法运算符，相当于 $i < j ? j : (i + 1)$ ，因此结果为 15，而不是预期的 16。

2. Preprocessor Directives (预处理指令)

Exercise 8

根据以下用法，评估我们的宏定义是否正确。你能发现哪些潜在问题？如何解决？

```
#define MAX(A, B) (A < B ? B : A)
```

```
int i = 10, j = 15;  
int k = MAX(i, i & j);
```

“ 宏展开后，k 的计算变为 $(i < i \& j ? i \& j : i)$ ，由于按位与运算符的优先级低于比较运算符，相当于 $((i < i) \& j) ? i \& j : i$ ，因此结果为 10。

2. Preprocessor Directives (预处理指令)

Exercise 8

根据以下用法，评估我们的宏定义是否正确。你能发现哪些潜在问题？如何解决？

```
#define MAX(A, B) ((A) < (B) ? (B) : (A))
```

```
int i = 10, j = 15;  
int k = MAX(i, ++j);
```

“ 宏展开后，`k` 的计算变为 `((i) < (++j) ? (++j) : (i))`，这会导致 `j` 被递增两次，因此结果为 17。

2. Preprocessor Directives (预处理指令)

2.7. #define: Function-Like Form

要点:

- 始终在 **宏参数** 和 **整个宏** 的周围加上 **括号**，以防止出现意外行为。
- **不要** 使用 `#define` 来替代函数。

2. Preprocessor Directives (预处理指令)

2.8. Conditional Inclusion: `#ifdef`, `#ifndef`, `#endif`

```
#ifdef MACRO_NAME  
    // Text block  
#endif
```

- `#ifdef` (或 `#ifndef`) 指令与 `#endif` 一起构成一个文本块。这些指令应放置在文件中，并且必须成对出现。
- 文本块是否包含在编译器的输入中取决于 `MACRO_NAME` 的定义：
 - 如果 `MACRO_NAME` 使用 `#define` 进行了定义，则 `#ifdef` 和 `#endif` 之间的文本将被包含，否则将被排除。
 - `#ifndef` 指令的作用方式相反。

2. Preprocessor Directives (预处理指令)

2.8. Conditional inclusion: `#if`, `#elif`, `#else`, `#endif`

- 由于 `macro_condition` 的值，将包含以下三个文本块中的且仅包含一个文本块：`Text block 1`、`Text block 2`(可选) 和 `Text block 3`(可选)。
- `expression` 是一个仅使用常量和通过 `#define` 指令定义的标识符的常量表达式。

```
#if macro_condition
    // Text block 1
#elif macro_condition
    // Text block 2
#else
    // Text block 3
#endif
```

2. Preprocessor Directives (预处理指令)

Example

```
#define ABCD 2
#include <stdio.h>

int main(void){

#ifdef ABCD
    printf("1: yes\n");    // printed

#else
    printf("1: no\n");

#endif
```

```
#ifndef ABCD
    printf("2: no1\n");
```

2. Preprocessor Directives (预处理指令)

Header Guards

- 如果你恰好在源文件中两次包含同一个头文件，例如：

```
#include "my_header.h"  
#include "my_header.h"
```

后果是灾难性的：`my_header.h` 中的所有定义（变量/函数）都会出现两次，这将导致程序崩溃。

- 你可能会认为这种情况永远不会发生。至少我们看看标准库是如何防止上述崩溃的？
 - 添加头文件保护符。建议在所有你的头文件中都这样做。

```
#ifndef HEADER_GUARDS  
#define HEADER_GUARDS  
#endif // HEADER_GUARDS
```

3. Variables

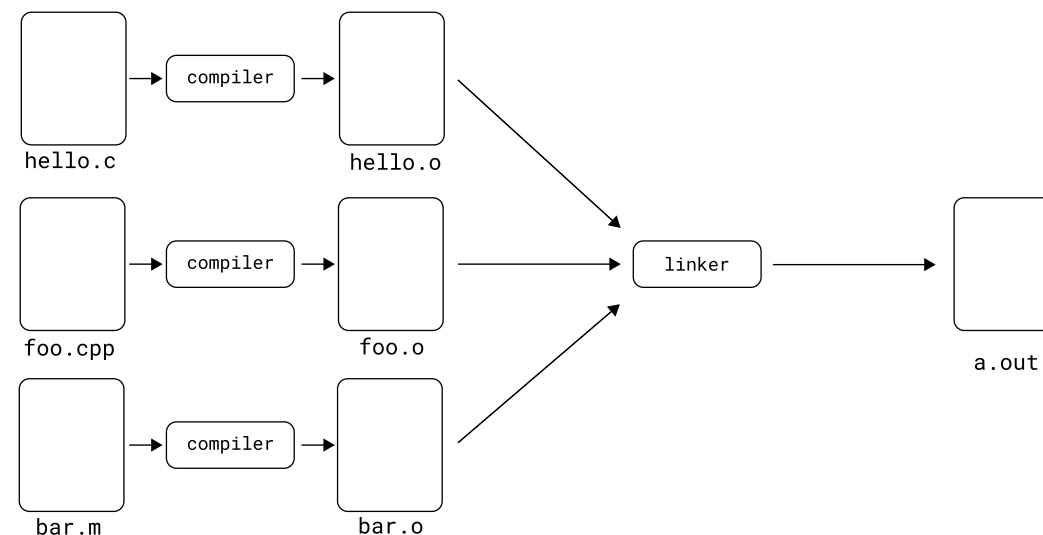
3. Variables

- 作用域 (Scope) and 生命周期 (Lifetime)
 - Storage duration
- 链接属性 (Linkage)
 - `static` keyword
 - `extern` keyword
- 隐式初始化 (Implicit initialization)

3. Variables

翻译单元 (Translation Unit)

- 在 C 和 C++ 编程术语中，翻译单元是 **C 或 C++ 编译器** 的最终 **输入**，编译器会根据此生成目标文件。
- 翻译单元通常由一个源文件组成，该源文件已经过 **C 预处理器** 处理。
- 这意味着头文件已被包含，条件编译部分（例如由 `#ifndef` 控制的部分）已被处理，宏也已展开。



3. Variables

作用域与生命周期 (Scope and Lifetime)

- **作用域**: 变量可以被访问的**代码**部分。
 - 在 C 程序中出现的每个 标识符 (identifier) -> 只能在源代码中的某些可能不连续的部分内可见 (即可使用)。
- **生命周期 ($\approx$ 存储期)**: 在程序执行期间, 一个 **对象** 是有效的 (存在、地址恒定、保留最后一次存储的值) 并可被使用的**时间长度**。
 - 在其生命周期之外访问一个对象会导致 **未定义行为**。

3. Variables

Example: Static local variable inside a function

```
#include <stdio.h>
void count_calls(void) {
    // static local variable
    static int count = 0;
    count++;
    printf("%d times\n", count);
}
int main(void) {
    count_calls(); // First call
    count_calls(); // Second call
    count_calls(); // Third call
    return 0;
}
```

- 作用域:
count 变量是在 count_calls() 函数内部声明的, 因此其作用域仅限于这个函数。它无法在该函数外部被直接访问或修改。
- 生命周期:
静态变量 count 的生命周期从程序开始运行时开始, 直到程序终止才会结束。它的值在函数调用之间是持续存在的。

```
called 1 times
called 2 times
called 3 times
```

3. Variables

Example: Accessing an object outside of its lifetime

```
int* dangling_pointer(void) {  
    int x = 5;  /  
    return &x;  
}  
  
int main(void) {  
    int *ptr = dangling_pointer();  
    // ptr points to a local variable that no longer exists  
    printf("%d\n", *ptr);  
    // Undefined behavior: accessing a variable after its lifetime  
    return 0;  
}
```

在 `main` 函数中访问 `dangling_pointer()` 的返回值的行为是 **未定义的行为**，因为在变量 `x` 的生命周期结束之后，`dangling_pointer` 才返回给其调用者 `main`。

3. Variables

- Example 1: Accessing global variable in a function while the variable is not in the local scope of the function.
- Example 2: Accessing variable outside its scope through pointer.

```
#include <stdio.h>

int global_x = 42;

void foo(void) {
    // local scope of foo() begins
    printf("%d\n", global_x);
}

int main(void) {
    foo();
    return 0;
}
```

```
#include <stdio.h>

void modify_value(int *ptr) {
    *ptr = 20;
    printf("%d\n", *ptr);
}

int main(void) {
    int x = 10;
    modify_value(&x);

    printf("%d\n", x);

    return 0;
}
```

3. Variables

Scope

当作用域重叠时，内部作用域中的声明会**隐藏外部作用域中的声明**。

- **块作用域**: 适用于函数体、`if`、`switch`、`for`、`while`、`do-while` 语句以及函数定义的参数列表。
 - 从声明点开始，到声明所在块或语句结束为止。
- **文件作用域**: 适用于在任何块或参数列表之外声明的标识符。
 - 从声明点开始，到**翻译单元**结尾结束。
- ***函数原型作用域**: `int f(int n, int a[n]);`
指在函数声明（而非定义）的参数列表中引入的名称。
 - 在函数声明符 `->` 结尾处结束。
- ***函数作用域, *声明点**: 超出类的作用域。

3. Variables

Variables: Nested Scopes

```
int a; // file scope of name a begins here
void f(void){
    int a = 1; // the block scope of the name a begins here; hides file-scope a
    {
        int a = 2; // the scope of the inner a begins here, outer a is hidden
        printf("%d\n", a); // inner a is in scope, prints 2
    } // the block scope of the inner a ends here

    printf("%d\n", a); // the outer a is in scope, prints 1
} // the scope of the outer a ends here
void g(int a);
// name a has function prototype scope; hides file-scope a;
// a is just a placeholder and is not accessible outside the prototype
```

3. Variables

Storage duration - limiting the object lifetime

- **自动存储持续时间**：仅在执行过程中的某些点存在
 - 应用于函数参数、非 `static` 的块作用域对象等。
 - 存储空间在进入声明该对象的代码块时分配，在退出该代码块时释放。
- **静态存储持续时间**：拥有整个程序的生命周期
 - 存储持续时间是程序的整个执行期间。
 - 存储在对象中的值**仅初始化一次**，在 `main` 函数之前完成。
 - 应用于所有声明为 `static` 的对象等。
- **动态分配存储持续时间**：其生命周期由程序员决定
 - 通过请求（动态内存分配函数）进行分配和释放。
- ***线程存储持续时间**：超出类的作用域

3. Variables

Scope and Storage duration

Type	Example	Scope	Duration
Local variable	<code>int x;</code>	Block	Automatic
Static local variable	<code>static int s_x;</code>	Block	Static
Function parameter	<code>void foo(int x)</code>	Block	Automatic
Internal global variable	<code>static int g_x;</code>	File	Static
External global variable	<code>int g_x;</code>	File	Static

3. Variables

Linkage: the ability to be referenced in other scope

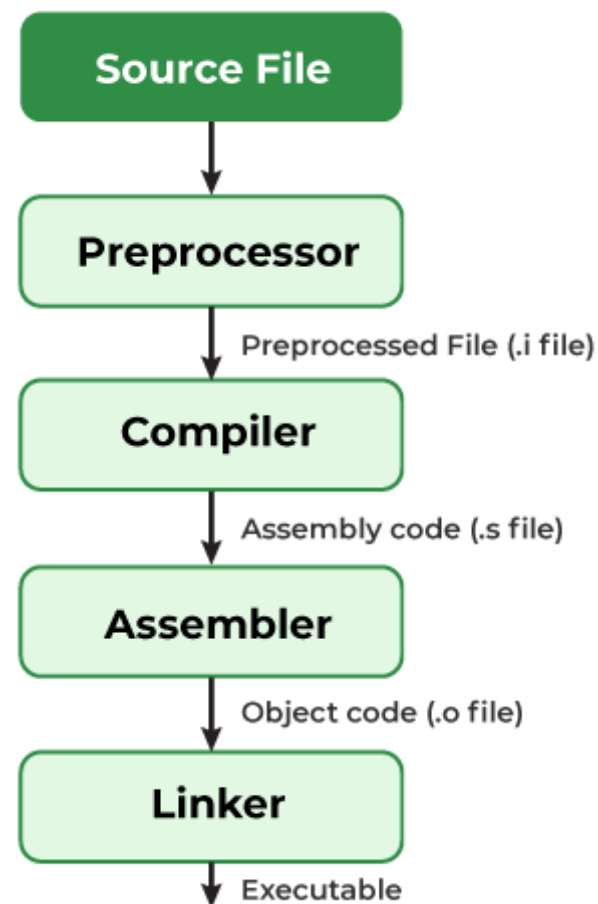
“链接属性 (Linkage)” 指的是一个标识符（变量或函数）在其他作用域中被引用的能力。以下是几种已知的链接类型：

- **无链接 (No linkage)**：该标识符只能在其自身的作用域内被引用。
 - 应用于所有函数参数和所有非 `extern` 的块作用域变量（包括声明为 `static` 的变量）。
- **内部链接 (Internal linkage)**：该标识符可以在当前翻译单元内被引用。
 - 应用于所有 `static` 的文件作用域标识符（包括函数和变量）。
- **外部链接 (External linkage)**：该标识符可以在其他任何翻译单元中被引用。
 - 应用于所有非 `static` 函数、所有 `extern` 变量（除非之前被声明为 `static`），以及所有非 `static` 文件作用域变量。

3. Variables

Linkage

- 在一个具有多个源文件的 C 程序中，每个文件都是逐一编译并链接成一个可执行文件。
- 在编译过程之前，变量的作用域定义了它的可见性和可访问性。
 - 编译器利用作用域的特性
- 链接过程中才涉及到链接属性的问题：
 - 变量是否应该在其他文件中被访问？
 - 变量是否应该仅限于在其声明文件内使用？



3. Variables

Scope, Storage duration and Linkage

Type	Example	Scope	Duration	Linkage
Local variable	<code>int x;</code>	Block	Automatic	None
Static local variable	<code>static int s_x;</code>	Block	Static	None
Function parameter	<code>void foo(int x)</code>	Block	Automatic	None
Internal global variable	<code>static int g_x;</code>	File	Static	Internal
External global variable	<code>int g_x;</code>	File	Static	External

3. Variables

Scope, Storage duration and Linkage

- 所有具有**内部或外部连接性**的对象都具有**静态存储持续时间**。
- **块作用域**变量默认情况下**没有链接属性**，且具有**自动存储持续时间**。
 - 局部变量的存储持续时间从进入块时开始，但在看到声明之前，变量不在作用域内，无法被访问。
- **文件作用域标识符**默认情况下具有**链接性**和**静态存储持续时间**。
 - 这些标识符的作用域仅限于它们自己的翻译单元，那么如何从其他翻译单元中引用它们？

☒ CppReference:

如果在多个作用域中声明了具有相同**标识符**的变量或函数，但是不能从所有作用域中引用，则会**生成该变量的多个实例**。

3. Variables

- For block-scope variables,
 - grant them static storage duration
 - ensure them to be initialized once

```
#include <stdio.h>
void counter() {
    static int count = 0;
    count++;
    printf("%d\n", count);
}
int main(void) {
    counter(); // Output: 1
    counter(); // Output: 2
    counter(); // Output: 3
    return 0;
}
```

- For file-scope identifiers (both functions and variables),
 - grant them static storage duration
 - grant them internal linkage

```
#include <stdio.h>
// File-scope static variable
static int file_scope_var = 42;
// File-scope static function
static void file_scope_function(void){
    printf("%d\n", file_scope_var);
}
int main(void){
    file_scope_function();
    return 0;
}
```

3. Variables

External Linkage: `extern` keyword

- 使用 `extern` 关键字声明的变量是外部变量，表明该变量在其他文件中定义。
- `extern` 说明符指定了 **静态存储持续时间** 和 **外部链接性**。

3. Variables

Example 1 of extern

File1: file1.c

```
#include <stdio.h>
// Definition of a global variable
int shared_variable = 42;
void display_variable(void) {
    printf("%d\n", shared_variable);
}
```

- 在 file1.c 中, shared_variable 为一个全局变量。
- 在 file2.c 中, 我们使用 extern 声明它, 这使我们可以访问并修改来自 file1.c 的 shared_variable。

File2: file2.c

```
#include <stdio.h>
extern int shared_variable;
void display_variable(void);
void modify_variable(void) {
    shared_variable = 100;
}
int main(void) {
    modify_variable();
    printf("%d\n", shared_variable);
    return 0;
}
```

Output:

42

100

3. Variables

File1: file1.c

```
#include <stdio.h>

// Definition of a global variable
int shared_variable = 42;

void function_with_extern() {
    // Declaration within block scope
    extern int shared_variable;
    // Block scope access
    printf("%d\n", shared_variable);
}
```

File2: file2.c

```
// Declaration of the external variable
extern int shared_variable;
void function_with_extern(void);

int main() {
    // accesses the variable
    function_with_extern();
    return 0;
}
```

Output: 42

3. Variables

Example 3 of extern

File1: file1.c

```
#include <stdio.h>

static int shared_variable = 42;

void display_variable(void) {
    printf("%d\n", shared_variable);
}
```

File2: file2.c

```
extern int shared_variable;
void display_variable(void);

void modify_variable(void) {
    shared_variable = 100;
}

int main(void) {
    modify_variable();
    return 0;
}
```

3. Variables

```
error: linker failed; symbol(s) not found.
```

- 静态全局变量只具有内部链接性，因此无法在其他翻译单元中访问它们。
- 即使在外部添加 `extern` 修饰的声明，其链接属性仍为内部。

3. Variables

Variables: Implicit Initialization

如果一个变量在声明时没有显式初始化：

- 具有**自动存储期**（例如，局部的非 `static` 变量）的对象将被初始化为**不确定值**。换句话说，它们是**未初始化的**。
- 对于具有静态存储期的对象（例如，全局或局部的 `static` 变量），它们会进行**空初始化** -> （在 C++ 中称为 零初始化 -> ）：
 - **指针被初始化为其类型的空指针值。**
 - 整数类型的对象被初始化为 0。
 - 浮点类型的对象被初始化为正零 (0.0)。
 - 数组的所有元素、结构体的所有成员以及联合体的第一个成员都会递归地进行空初始化，并且所有填充位都会被初始化为零。

3. Variables

Exercise 9

“ Read the following code segments. Tell whether it can compile or not, if can, what's the output?

3. Variables

Exercise 9.1

```
#include <stdio.h>

int main(void){
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

Exercise 9.2

```
#include <stdio.h>

int n = 0;

int main(void) {
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

3. Variables

Exercise 9.1

```
#include <stdio.h>

int main(void){
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

“ 无法编译，因为 `n` 未声明。

Exercise 9.2

```
#include <stdio.h>

int n = 0;

int main(void) {
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

“ 可以编译，输出 5。

3. Variables

Exercise 9.3

```
#include <stdio.h>

int n = 0;

int main(void) {
    int n;
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

Exercise 9.4

```
#include <stdio.h>

int n = 0;

int main(void){
    int n = 1;

    n += 5;
    printf("%d\n", n);

    return 0;
}
```

3. Variables

Exercise 9.3

```
#include <stdio.h>

int n = 0;

int main(void) {
    int n;
    n += 5;
    printf("%d\n", n);

    return 0;
}
```

“ 可以编译，但是产生**未定义的行为**，因为局部变量 `n` 未初始化。

Exercise 9.4

```
#include <stdio.h>

int n = 0;

int main(void){
    int n = 1;

    n += 5;
    printf("%d\n", n);

    return 0;
}
```

“ 可以编译，输出 6。

3. Variables

Exercise 9.5

```
#include <stdio.h>

int n = 0;

int main(void) {
    extern int n;

    n += 5;
    printf("%d\n", n);

    return 0;
}
```

Exercise 9.6

```
#include <stdio.h>

int total(int x) {
    static int total = 0;

    total += x;
    return total;
}

int main(void) {
    total(5);
    total(10);
    printf("%d\n", total(15));
    return 0;
}
```

3. Variables

Exercise 9.5

```
#include <stdio.h>

int n = 0;

int main(void) {
    extern int n;

    n += 5;
    printf("%d\n", n);

    return 0;
}
```

“ 可以编译，输出 5。

Exercise 9.6

```
#include <stdio.h>
int total(int x) {
    static int total = 0;

    total += x;
    return total;
}
int main(void) {
    total(5);
    total(10);
    printf("%d\n", total(15));
    return 0;
}
```

“ 可以编译，输出 30。

3. Variables

library.h:

```
#ifndef LIBRARY_H
#define LIBRARY_H

int net_price(int price, int discount);
static int validate(int n);

#endif
```

driver.c:

```
#include "library.h"
int main(void) {
    int i, j;
    i = validate(-5);
```

library.c:

```
#include "library.h"

int net_price(int price, int discount) {
    return validate(price - discount);
}

static int validate(int n) {
    if (n < 0)
        return 0;
    else
        return n;
}
```

“可以编译，但无法链接，因为 `validate` 函数具有

3. Variables

Exercise 10

- ☒ 确定头文件和源文件能否分别进行编译。如果可以，它们能够链接在一起吗？

3. Variables

Exercise 10.1

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

int acct_status;

#endif
```

account.c:

```
#include "account.h"
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

3. Variables

Exercise 10.1

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

int acct_status;

#endif
```

account.c:

```
#include "account.h"
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

“能够编译: account.h 在两个翻译单元中都有包含
无法链接: 因为 acct_status 在两个翻译单元中都已定义

3. Variables

Exercise 10.2

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

// Don't define acct_status
// since that caused the problem

#endif
```

account.c:

```
#include "account.h"
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

3. Variables

Exercise 10.2

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

// Don't define acct_status
// since that caused the problem

#endif
```

account.c:

```
#include "account.h"
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

“ 不能编译，你会得到类似 “'acct_status' was not declared in this scope.” 的报错信息。

3. Variables

Exercise 10.3

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

// Don't define acct_status

#endif
```

account.c:

```
#include "account.h"
int acct_status;
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

3. Variables

Exercise 10.3

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

// Don't define acct_status

#endif
```

account.c:

```
#include "account.h"
int acct_status;
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

“ account.c 可以编译但 driver.c 不行。

3. Variables

Exercise 10.4

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

extern int acct_status;

#endif
```

account.c:

```
#include "account.h"

// Use acct_status
```

driver.c:

```
#include "account.h"

// Use acct_status
```

3. Variables

Exercise 10.4

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

extern int acct_status;

#endif
```

account.c:

```
#include "account.h"
// Use acct_status
```

driver.c:

```
#include "account.h"
// Use acct_status
```

“ 它们都能被编译: `acct_status` 已经在 `account.h` 中声明。

无法链接: 因为 `acct_status` 只有声明, 没有定义。

3. Variables

Exercise 10.5

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

extern int acct_status;

#endif
```

account.c:

```
#include "account.h"

int acct_status = ACCT_OK;

// Use acct_status
```

driver.c:

```
#include "account.h"

// Use acct_status
```

3. Variables

Exercise 10.5

account.h:

```
#ifndef ACCOUNT_H
#define ACCOUNT_H

#define ACCT_OK      0
#define ACCT_INVALID 1
#define ACCT_CLOSED  2

extern int acct_status;

#endif
```

account.c:

```
#include "account.h"

int acct_status = ACCT_OK;

// Use acct_status
```

driver.c:

```
#include "account.h"

// Use acct_status
```

Good luck and enjoy programming!